

HMalloc: A Hybrid, Scalable, and Lock-free Memory Allocator for Multi-threaded Applications

Tianlin Li

National University of Defense Technology
Changsha, China
ltl@mail.ustc.edu.cn

Wenjie Tang

National University of Defense Technology
Changsha, China
tangwenjie@nudt.edu.cn

Yiping Yao

National University of Defense Technology
Changsha, China
yyp@nudt.edu.cn

Zhongwei Lin

National University of Defense Technology
Hefei, China
zwlin@nudt.edu.cn

Abstract—An efficient multi-threaded memory allocator has great impact on performance of applications with frequent memory allocation and deallocation operations. Currently, most popular memory allocators ignore the difference between thread local and shared memory, and manage them in a unified manner, which cannot make good use of the spatiotemporal locality of memory access. To solve this problem, this paper proposes a hybrid and lock-free multi-threaded memory allocator, named HMalloc. The allocator separates local memory from shared memory. There is no false sharing and lock contentions in local memory allocation and deallocation process. Moreover, coalescence-free is used to optimize this process. Further, a flag-based shared memory allocation and deallocation method is proposed to achieve lock-free shared memory management. Experimental results show that HMalloc can achieve significant performance improvement when compared with existing well-known memory allocators.

Keywords—hybrid, lock-free, scalable, multi-threaded, memory allocator

I. INTRODUCTION

Memory allocation and deallocation are the most basic operations in multi-threaded applications, such as malloc/free functions in a C or C++ application. Hence, scalable and efficient multi-threaded memory allocators are necessary. Existing multi-threaded memory allocators mostly focus on implementing lock-free and scalable memory management, such as Hoard [2], SSMalloc [5], and SFMalloc [7]. However, they ignore the difference between local and shared memory. Besides, lock-free allocation and deallocation of shared memory depends on compare and swap (CAS) atomic instructions, such as NBMalloc [4], SSMalloc. However, frequent atomic operations will degrade the performance of multi-threaded application.

A completely lock-free memory allocator does not exist because multiple threads will contend on operating system (OS) kernel when they perform VM management operations simultaneously, such as mmap and munmap operations. Thence, an excellent lock-free memory allocator should minimize the frequency of allocating raw memory blocks from OS directly.

This paper proposes a hybrid, scalable, and lock-free multi-threaded memory allocator, named HMalloc. It separates thread local memory from shared memory. Thread local and shared memory are managed separately. The management of local memory can make full use of memory space. Besides, coalescence-free method is adopted to free local memory. To minimize the frequency of mmap operations, non-fixed-size memory superblocks are

introduced. A flag-based shared memory management method is proposed to implement lock-free allocation and deallocation for shared memory, achieving remote-free process without lock contention.

The rest of this paper is organized as follows. Section II describes the related work. Section III presents the design and implementation of HMalloc. Experimental results and analysis are provided in Section IV. Finally, Section V concludes the paper.

II. RELATED WORK

With the development of multithreading technology, scalable and efficient multi-threaded memory allocators have attracted a lot of attention. There have been many excellent multi-threaded memory allocators, such as Hoard, SSMalloc, and so on.

Hoard is a very typical scalable memory allocator for multi-threaded applications. However, it is not lock-free. When multiple threads access freed superblock list in the global heap at the same time, lock contentions will happen. Besides, heap contentions will also happen when a thread allocates memory from one heap and a number of threads free memory that belongs to the heap. Based on Hoard, Michael implemented a lock-free dynamic memory allocator by introducing CAS atomic instructions to implement lock-free freed superblock list [3].

Liu et al. designed a SSMalloc memory allocator. The global freed chunk list is also implemented to be lock-free by CAS atomic instructions. This allocator mainly adopted the following three improvements: 1) reducing the memory management overhead through minimized critical path; 2) reducing VM management system calls; 3) adopting the lock-free and most wait-free algorithms to optimize the memory management process. Leite et al. also proposed a lock-free allocator, which still depends on CAS instructions to implement the lock-free memory management process [1].

Frequent CAS will lead to some problems [6]. For example, there exists a critical section which begins by reading a shared variable and the variable is updated by CAS instructions. When multiple threads execute this critical section simultaneously, one thread update the variable first and other lagging threads will repeat the section. This leads to a lot of extra overhead. Seo et al. proposed a lock-free and mostly synchronization-free dynamic memory allocator named SFMalloc. They designed a lock-free memory stack to handle shared memory. This allocator has some disadvantages. First, the critical path of is long. Moreover, the lock-free stack needs large amount of atomic operations.

In summary, existing multi-threaded memory allocators usually ignore the difference between thread local memory and shared memory. Besides, the implementation of lock-free must rely on atomic instructions.

III. DESIGN AND IMPLEMENTATION OF HMALLOC

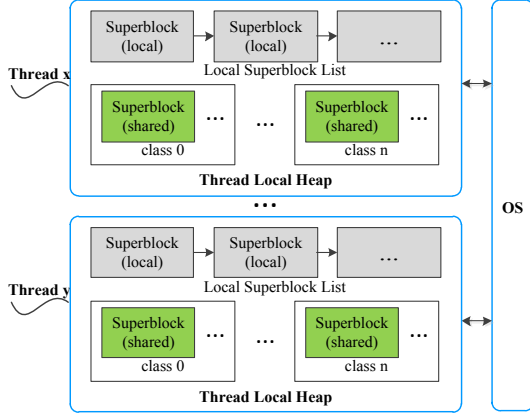


Fig. 1 HMalloc architecture.

Fig. 1 presents the architecture of HMalloc. Each thread owns a local heap to perform memory allocation and deallocation. The memory blocks in a local heap only come from OS. Assuming that a thread will consume a peak memory space of M in its lifecycle, N -threads application will cost at most $O(N*M)$ memory size. Hence, memory blowup will not happen even though there is no global heap.

Superblock is the basic unit of memory allocated from OS. Local superblock is only responsible for thread local memory allocation, while shared superblock is used for shared memory allocation. Each local heap consists of a local superblock list and a number of shared superblock lists classified by different size classes. Each superblock only performs memory allocation requests by its owner thread. A local superblock only perform the deallocation requests from its owner thread, while a shared superblock will perform deallocation requests from multiple threads. Here, remote-free method is needed. If a memory block in a shared superblock is freed by a remote thread, the freed memory block still belong to its owner thread and can be reused for later memory allocation requests.

A. Superblock Structures

Fig. 2 represents the data structure of a local superblock. Local superblock consists of a superblock header and many objects of different sizes. The size of an object in a local superblock is variable and equals to the corresponding memory requested size. HMalloc adopts two sizes of local superblock, 64KB and 4MB. Usually, a local superblock of 64KB can satisfy all memory allocation requests. When the local memory request of a thread exceeds 64KB, a local superblock of 4MB size is allocated from OS to satisfy the memory allocation request. The superblock header maintains the key information of the owner superblock, the owner thread id, the starting address of the first sub-block (StartAddr), the number of allocated blocks (AllocatedNum), the number of freed blocks (FreedNum), and the address for next memory allocation request (CurrentAddr). When AllocatedNum equals to FreedNum, Coalescence-free method is triggered to reset these parameters.

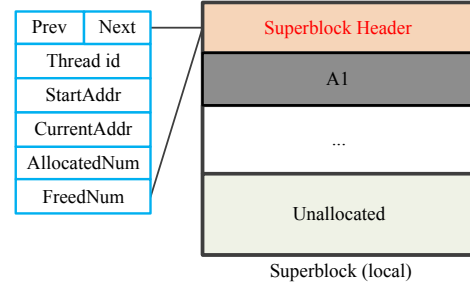


Fig. 2. Thread local superblock.

Fig. 3 shows the structure of thread shared superblock. shared superblock also has two sizes, 64KB and 4 MB. But it contains many objects of same size. In multi-threaded applications, shared memory blocks are usually used to store messages between threads, and messages are usually fixed size. To avoid false sharing, the requested sizes are aligned to multiples of cache line size, and shared superblocks are classified according to multiples of 64B, class 0 is 64B, class 1 is 128B and so on. Shared superblocks of same class are stored in same list. To achieve lock-free allocation and deallocation of shared memory, we proposed a flag-based lock-free shared memory remote-free method. Each memory block reserves 8 bytes to store a flag value. The flag value indicates whether the memory block is allocated or freed. The reason why a flag occupies 8 byte is that the OS owns a 64-bit address space. When a memory block is allocated, the corresponding flag value will be set to 1, and reset to 0 when the block is freed by another thread. Obviously, the allocation of a shared memory must precede the deallocation. Hence, the method is lock-free. When the flag value of a shared memory block is reset, the block can be reused by its owner thread for later allocation requests.

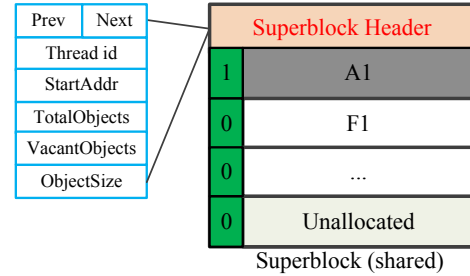


Fig. 3. Thread shard superblock.

B. Allocation Process

Algorithm 1: Thread local memory Allocation Process

```

1  obtain the thread local heap and current superblock
2  if the requested size > the half size of current superblock
3    allocate memory from OS and return;
4  else
5    if current superblock can satisfy the request
6      update CurrentAddr, AllocatedNum;
7      return;
8    else
9      allocate a new superblock;
10     perform the allocation;
11     return;
12  end

```

Fig. 4. Thread local memory allocation process.

Fig. 4 shows the thread local memory allocation process. When a thread performs a local memory request, it should obtain the local heap and current superblock first. When the requested size exceeds the half size of current superblock, the allocation will be satisfied by OS kernel. Otherwise, the thread will judge whether the left space of current superblock is larger than the requested size. If true, CurrentAddr can be used as the return address directly. Besides, CurrentAddr, AllocatedNum will be updated. If not, the thread should allocate a new superblock. The new superblock will become current superblock, and the old one will be inserted to LSL.

Algorithm 2: Thread shared memory Allocation Process

```

1  obtain the thread local heap and current superblock
2  if the requested > 32 KB
3    allocate memory from OS directly;
4  else
5    if current superblock can satisfy the request
6      update the flag value and return the address;
7    else
8      allocate a new superblock;
9      perform the allocation;
10   return;
11 end

```

Fig. 5. Thread share memory allocation process.

Fig. 5 shows the shared memory allocation process. When a thread produce a shared memory request, it should obtain the local heap and current superblock first. Next, it will allocate memory block from OS directly if the requested size exceeds 32KB. Otherwise, it will obtain a shared superblock of appropriate size class. Then thread can obtain the available address and set the flag value to 1.

In summary, there is no lock contentions or atomic instructions in thread local or shared memory allocation process. All the local superblocks in same local heap are only accessed by its owner thread, and the shared superblocks only satisfy the allocation requests from its owner thread. What's more important, there is no stack structure in HMalloc. Hence, ABA problem [7] will not happen.

C. Deallocation Process

The deallocation process of thread local memory is presented in Fig. 6. If the memory block comes from OS directly, the thread will free it to OS directly. Otherwise, the FreedNum will be updated. If FreedNum equals to AllocatedNum, coalescence-free will be triggered.

Algorithm 3: Thread local memory Deallocation Process

```

1  if the memory block is allocated from OS
2    free it to OS directly;
3  else
4    update FreedNum;
5    if FreedNum equals to AllocatedNum
6      perform coalescence-free;
7    end
8  end

```

Fig. 6. Local memory deallocation process.

As shown in Fig. 7, the address of the corresponding flag can be obtained according to the address ptr to be freed. Then, the thread directly set the flag value to 0, indicating that the memory block has been freed. Through updating the flag value, the allocation and deallocation of share memory blocks will be lock-free. Moreover, the critical path is minimized.

Traditional remote-free methods usually depend on CAS atomic instructions to implement lock-free deallocation process. Compared with traditional allocators, flag-based remote-free method is much simpler and has minimized critical path.

Algorithm 4: Thread shared memory Deallocation Process

Data: *ptr*, the address of the freed memory block

```

1  if the memory block is allocated from OS
2    free it to OS directly;
3  else
4    set the flag value to 0;
5  end

```

Fig. 7. Shared memory deallocation process.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

In order to validate the performance and scalability of HMalloc allocator, we compare HMalloc to several well-known multi-threaded allocators, SFMalloc, SSMalloc and the allocator in glibc (2.12). Four famous benchmarks are used to test these allocators, they are Threadtest, Linux-Scalability, Largetest and Producer-Consumer. They can verify the performance of these allocators in three aspects, speed and scalability, large memory blocks and producer-consumer patterns.

A. Speed and Scalability

In Threadtest benchmark, N threads allocate 100,000/N 64B blocks and then free all of them. This process is repeated 5,000 times. Fig. 8(a) presents the experimental results when tested by ThreadTest benchmark. The performance of allocator in glibc plays a role of baseline. Compared with SSMalloc, HMalloc can achieve about 5%-18% performance improvement, and about 44% - 51% performance improvement can be achieved when compared with SFMalloc allocator. These results demonstrate that the coalescence-free method can accelerate the allocation and deallocation of local memory obviously.

In Linux-Scalability benchmark, each thread allocates 512B blocks for 1,000,000 times and then free all of them. As shown in Fig. 8(b), HMalloc performs better than SSMalloc when the thread number is less than 6 or larger than 15. Besides, HMalloc performs worse than SSMalloc. Moreover, HMalloc can achieve about 35%-300% performance improvement when compared with SFMalloc. The reason why HMalloc performs worse than SSMalloc is that Linux-Scalability benchmark consumes large amount of memory. One thread will need about 500 MB. Hence, each thread will allocate about at least 125 superblocks in HMalloc allocator. HMalloc needs to scan the local superblock list again when current superblock is lack of memory space. Traversing the list will consume a certain amount of time. Fortunately, this situation rarely occurs in practical multi-threaded applications. Through these two benchmarks, the excellent performance and scalability of HMalloc allocator is verified.

B. Large Memory Blocks

The Largetest benchmark is used to test the performance of allocators when performing large amount of memory allocation requests with large requested sizes. In this benchmark, each thread allocate 100 memory blocks of 64B and allocate a large memory block of 64 KB, then free all of

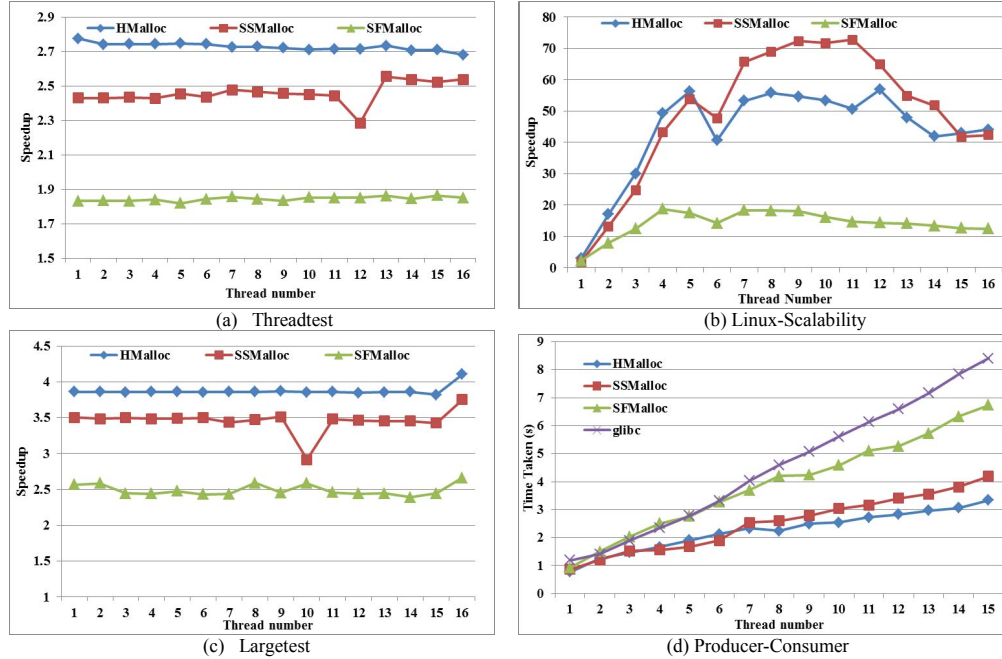


Fig. 8. Experimental results.

them. This process is repeated 1,000,000 times. As shown in Fig. 8(c), HMalloc can achieve about 10% performance improvement when compared with SSMalloc, and more than 50% performance improvement when compared with SFMalloc allocator. The experimental results validate that HMalloc allocator has better stability when performs memory requests of different sizes.

C. Producer-Consumer

In this benchmark, a producer thread is responsible for allocating shared memory blocks of 8B, and other consumer threads free these memory blocks. This process is repeated 1,000,000 times. Fig. 8(d) shows the execution time of the four allocators. We can find that HMalloc performs better than SSMalloc in most cases, and the advantage becomes greater when thread number increases. HMalloc can achieve about 10% - 25% performance improvement when compared with SSMalloc, and about 20% - 106% performance improvement when compared with SFMalloc. The reason why HMalloc performs worse than SSMalloc when thread number is larger than 3 and less than 6 is that the frequency of calling CAS atomic instructions is less when the thread number is small. When the number of threads increase, CAS atomic instructions will be called frequently, which degrades the performance of SSMalloc obviously. Through Producer-Consumer benchmark, the flag-based shared memory allocation and deallocation method is verified to be efficient. This method not only implements lock-free allocation and deallocation process, but also greatly reduces the critical execution path.

V. CONCLUSION

This paper proposes an efficient multi-threaded memory allocator, named HMalloc. It innovatively separates thread local memory from shared memory. There is no false sharing and lock contentions in allocation and deallocation process

of local memory. Moreover, coalescence-free is adopted to optimize local memory deallocation process. To reduce the frequency of mmap operations and lock contentions on OS level, HMalloc utilizes non-fixed-size superblocks. HMalloc adopts a flag-based memory management method to implement lock-free allocation and deallocation of shared memory. Experimental results show that HMalloc can achieve significant performance improvement when compared with traditional multi-threaded memory allocators.

ACKNOWLEDGMENT

The authors appreciate the support from National Natural Science Foundation of China (No. 61702527 and 61802422).

REFERENCES

- [1] R. Leite and R. Rocha, "LRMalloc: A Modern and Competitive Lock-Free Dynamic Memory Allocator," VECPAR 2018, pp. 230-243, 2018.
- [2] E. D. Berger, K. S. McKinley, R. D. Blumofe, and P. R. Wilson, "Hoard: A Scalable Memory Allocator for Multi-threaded Applications," ASPLOS 2000, Cambridge, MA USA.
- [3] M. M. Michael, "Scalable Lock-Free Dynamic Memory Allocation," PLDI 2004, Washington, DC, USA, June 2004.
- [4] A. Gidenstam, M. Papatriantafyllou, and P. Tsigas, "NBMALLOC: Allocating Memory in a Lock-Free Manner," Algorithmica vol. 58, pp. 304-338, 2010.
- [5] R. Liu, H. Chen, "SSMalloc: A Low-latency, Locality-conscious Memory Allocator with Stable Performance Scalability," APSys 2012, Seoul, S. Korea, July, 2012.
- [6] X. Huang, C. I. Rodrigues, S. Jones, I. Buck, and W. Hwu, "XMMalloc: A Scalable Lock-free Dynamic Memory Allocator for Many-core Machines," 2010 10th IEEE International Conference on Computer and Information Technology, pp. 1134-1139, 2010.
- [7] S. Seo, J. Kim, and J. Lee, "SFMalloc: A Lock-Free and Mostly Synchronization-Free Dynamic Memory Allocator for Manycores," 2011 International Conference on Parallel Architectures and Compilation Techniques, pp. 253-263, 2011.